

Indice y resumen

DOCUMENTO DE ESPECIFICACIÓN DE REQUERIMIENTOS Y ARQUITECTURA (DERA)

Proyecto: Plataforma Web E-commerce y Panel de Gestión – WEEKSPORT

Autor: Augusto Enrique Pugliano

Estado: Fase 1 (Hacia el MVP)

1. VISIÓN GENERAL Y CONTEXTO DEL NEGOCIO

- **1.1. Propósito del Sistema:** Definición del objetivo de digitalizar el catálogo y la separación entre el e-commerce (clientes) y el backoffice (administración).
- **1.2. Modelo Operativo y Reglas de Negocio:** Explicación del entorno transaccional asincrónico y la decisión de usar un flujo de conciliación manual para evitar falsos positivos en el stock.
- **1.3. Objetivos de Ingeniería (Métricas Clave):**
 - 1.3.1. Meta de costos operativos fijos (\$0 USD).
 - 1.3.2. Estándares de seguridad (Aislamiento de datos, JWT, RLS).
 - 1.3.3. Rendimiento esperado (Latencia y soporte de picos de concurrencia).

2. ARQUITECTURA TECNOLÓGICA (STACK DESACOPLADO)

- **2.1. Capa Frontend (Interfaz de Usuario):** Especificaciones sobre Next.js, Tailwind CSS y la estrategia de renderizado.
- **2.2. Alojamiento y Despliegue:** Configuración en Vercel y gestión de dominios.
- **2.3. Capa Backend y Persistencia:** Uso de Supabase como motor PostgreSQL relacional.
- **2.4. Servicios de Infraestructura Adicionales:**
 - 2.4.1. Módulo de Autenticación (Supabase Auth).
 - 2.4.2. Almacenamiento de Medios (Supabase Storage).

3. ESPECIFICACIÓN DE REQUERIMIENTOS

- **3.1. Módulo de Cliente (E-commerce Público):**
 - 3.1.1. Catálogo Dinámico y Filtros Avanzados (RF-01, RF-02).
 - 3.1.2. Gestión de Carrito Local (RF-03, RF-04).
 - 3.1.3. Integración de Checkout vía WhatsApp (RF-05).
- **3.2. Módulo de Administrador (Backoffice):**
 - 3.2.1. Autenticación y Control de Accesos (RF-06).
 - 3.2.2. Panel CRUD de Inventario y Alertas de Stock (RF-07).
 - 3.2.3. Carga y Gestión de Imágenes (RF-08).
 - 3.2.4. Motor de Conciliación y Deducción de Stock (RF-09).

- **3.3. Requerimientos No Funcionales (RNF):** Políticas de seguridad, estrategia de caché (ISR) y diseño Mobile-First.

4. MODELO DE DATOS Y ESTADO DE LA BASE DE DATOS

- **4.1. Evolución del Esquema Relacional:** Historial de la migración del esquema original a la normalización actual.
- **4.2. Diccionario de Datos (Entidades Principales):**
 - 4.2.1. Entidad: **categorias** (Gestión de agrupaciones y slugs).
 - 4.2.2. Entidad: **productos** (Propiedades comunes y relaciones).
 - 4.2.3. Entidad: **variantes_stock** (Matriz de dimensiones físicas y precios).
- **4.3. Estrategia de Indexación:** Índices B-Tree implementados para optimización de consultas.

5. SEGURIDAD Y CONFIGURACIÓN DE INFRAESTRUCTURA

- **5.1. Políticas de Seguridad a Nivel de Fila (RLS):**
 - 5.1.1. Permisos del Rol Público (**anon**).
 - 5.1.2. Permisos del Rol Administrativo (**authenticated**).
- **5.2. Infraestructura de Storage:** Configuración, límites y políticas del bucket **productos-imagenes**.
- **5.3. Gestión del Espacio de Trabajo Local:** Estructura de directorios, manejo de variables de entorno y exclusiones estratégicas.

6. ESPECIFICACIÓN DE MÓDULOS CRÍTICOS Y LÓGICA DE NEGOCIO

- **6.1. Algoritmo de Checkout para WhatsApp:** Lógica de compilación, sanitización de inputs y generación de URL.
- **6.2. Arquitectura de Autenticación y Sesiones:** Resolución de conflictos de persistencia de sesión entre cliente y servidor.

7. CONTROL DE CALIDAD (QA) Y REGISTRO DE AUDITORÍA

- **7.1. Matriz de Verificación (Casos de Prueba):** Tabla de control para certificar la compilación, aislamiento de credenciales y sintaxis SQL.
- **7.2. Registro Histórico de Sesiones de Desarrollo:** Resumen de intervenciones, agentes involucrados y divergencias detectadas.
- **7.3. Registro de Errores Resueltos:** Documentación de soluciones aplicadas (Ej: Loop de redirección, rutas duplicadas, problemas de Server/Client components).

8. ROADMAP Y PLAN DE ACCIÓN HACIA EL MVP (Tareas Pendientes)

- **8.1. Resolución de Deuda Técnica Inmediata:** Diagnóstico y corrección del bug bloqueante en la creación de productos (conflicto de restricción **NOT NULL** en la columna obsoleta **categoria**).
- **8.2. Limpieza de Esquema de Datos:** Evaluación y ejecución de la eliminación definitiva de columnas deprecadas en PostgreSQL.
- **8.3. Verificación End-to-End (E2E):** Pruebas de integración del flujo completo, desde la creación en el panel hasta la deducción transaccional (RF-09).
- **8.4. Refactorización de Código:** Actualización del middleware hacia los estándares más recientes de Next.js.
- **8.5. Integración de Datos de Producción:** Reemplazo de datos de prueba por activos reales (logo definitivo, paleta de colores de la marca, catálogo inicial y número comercial de WhatsApp).

Vision general y contexto

1. VISIÓN GENERAL Y CONTEXTO DEL NEGOCIO

1.1. Historia y Naturaleza de la Organización

WEEKSPORT es un emprendimiento familiar de indumentaria y artículos deportivos radicado en Quilmes Oeste. El negocio comercializa tanto prendas de elaboración propia (fabricación manual) como artículos de reventa, abarcando catálogos de temporada (verano e invierno) y accesorios complementarios (toallas, botellas de agua, riñoneras, entre otros).

A nivel operativo, la organización cuenta con una estructura compacta de dos roles principales: la dueña fundadora (encargada de la dirección y logística) y una empleada que asume funciones de atención al cliente y Community Manager. Hasta el inicio de este proyecto, el nivel de digitalización comercial de la marca era incipiente, concentrando el volumen de ventas casi exclusivamente en el mostrador del local físico y utilizando Instagram de forma pasiva.

El propósito principal de esta plataforma es dotar a la marca de una identidad digital profesional para potenciar sus redes sociales, implementando una solución de software de dos caras: un e-commerce público (catálogo) para la captación de clientes y un backoffice privado para la administración interna.

1.2. El Problema Operativo (Puntos de Dolor)

Previo a la implementación de este sistema, la administración del inventario se realizaba íntegramente a través de herramientas ofimáticas genéricas (Google Sheets). A medida que el catálogo creció, este enfoque comenzó a presentar limitaciones críticas:

- **Fricción en la consulta:** Revisar la disponibilidad física de variantes específicas (ej. un talle y color particular) cruzando filas y columnas resultaba un proceso visualmente tedioso y propenso a errores humanos.
- **Inconsistencia de datos (Stock Fantasma):** La dependencia de la carga y descarga manual en una planilla no relacional generaba desajustes operativos temporales. Era frecuente que tras concretar una venta en el local, se omitiera la actualización en el Excel, resultando en un stock teórico que no reflejaba la realidad física (ofreciendo productos que ya no existían o invisibilizando los que sí).

1.3. Modelo Operativo y Reglas de Negocio (Checkout Asincrónico)

Dada la escala del negocio (tráfico web proyectado menor a 10 usuarios concurrentes diarios) y la naturaleza de cercanía con su clientela de barrio, el sistema fue diseñado evitando complejidades innecesarias.

Se optó deliberadamente por **no integrar pasarelas de pago transaccionales** (como Mercado Pago o Stripe). En su lugar, el modelo de negocio opera bajo un entorno

transaccional asincrónico (pagos diferidos mediante transferencia bancaria o efectivo). El sistema implementa un "Checkout por Mensajería":

1. El cliente navega el catálogo web y preselecciona los productos en un carrito virtual.
2. Al confirmar, el sistema captura la orden, empaqueta los datos y redirige al usuario a WhatsApp mediante un enlace dinámico (wa.me).
3. El enlace inyecta un mensaje pre-formateado y detallado en el chat del cliente, conectándolo directamente con la Community Manager para pactar la entrega y el pago.
4. Para evitar bloqueos de inventario por carritos abandonados (falsos positivos), el stock físico **solo se descuenta de la base de datos** cuando una de las administradoras valida fehacientemente el pago en el backoffice, a través de un flujo de conciliación manual.

1.4. Justificación Arquitectónica (Perspectiva de Ingeniería)

El desarrollo de este sistema representó un desafío técnico claro: entregar software con estándares de calidad empresarial a un negocio de barrio, manteniendo un costo de infraestructura en servidores de **\$0 USD fijos**.

En lugar de recurrir a arquitecturas de backend tradicionales (como construir una API REST monolítica con Java/Spring Boot o Node.js), las cuales implican tiempos de desarrollo (Time-to-Market) prolongados, mantenimiento constante de endpoints y costos de hosting, se adoptó un paradigma **Serverless y Backend-as-a-Service (BaaS)**.

- **Persistencia y Backend Delegado (Supabase):** Se delegó la capa de backend a Supabase, obteniendo una base de datos PostgreSQL completa con autenticación (JWT) y almacenamiento en la nube (Storage) listos para usar. Si bien el uso de bases de datos remotas alojadas en datacenters externos (ej. región Brasil) puede presentar una latencia marginal de milisegundos en el panel de administración frente a la respuesta local de una hoja de Excel, el beneficio neto en integridad referencial, seguridad de concurrencia y consistencia de datos justifica ampliamente este *trade-off*.
- **Frontend Desacoplado (Next.js):** El uso de Next.js (App Router) alojado en Vercel permite proteger la base de datos de lecturas repetitivas y entregar el catálogo a los usuarios con velocidades de carga casi instantáneas, aprovechando las redes de distribución de contenido (CDN) gratuitas.

Esta decisión demuestra un enfoque pragmático de ingeniería, priorizando la resolución ágil de un problema de negocio real utilizando las herramientas más modernas del mercado, sin sacrificar rendimiento ni incurrir en deuda técnica.

1.5. Alcance Final (Definición del MVP)

El alcance general de este proyecto se delimita estrictamente al desarrollo e implementación de la Fase 1 (MVP). El producto se considera completo con el correcto funcionamiento del catálogo web, su derivación efectiva a WhatsApp, y la capacidad del

Panel de Control para realizar operaciones CRUD (Crear, Leer, Actualizar, Borrar) sobre el inventario.

No se proyecta la evolución del sistema hacia funcionalidades como la creación de perfiles de usuario, bases de datos de clientes, o automatización de envíos logísticos, ya que el alcance actual de la aplicación cubre eficientemente la necesidad máxima operativa esperada por las administradoras de WEEKSPORT.

Arquitectura tecnologica

2. ARQUITECTURA TECNOLÓGICA (STACK DESACOPLADO)

El sistema se cimenta sobre una arquitectura moderna desacoplada basada en el paradigma JAMstack, priorizando la simplicidad estructural, la eficiencia en el tamaño del paquete de distribución y un costo de infraestructura de \$0 USD fijos. Al encontrarse el proyecto en su Fase 1 de desarrollo, se ha optado por mantener un entorno limpio de dependencias de terceros para asegurar un control absoluto sobre el flujo de datos.

2.1. Capa Frontend (Interfaz de Usuario)

La aplicación está construida utilizando el framework **Next.js 15+**, adoptando su arquitectura nativa de **App Router** para la estructuración de rutas, layouts jerárquicos y la separación eficiente de responsabilidades en componentes de servidor y cliente.

- **Estilizado y Diseño Visual:** Se implementa de forma nativa **Tailwind CSS v4** ([@tailwindcss/postcss](#)). La elección de la versión 4 responde a la necesidad de aprovechar el nuevo motor de compilación optimizado, que reduce drásticamente los tiempos de empaquetado y mejora el rendimiento de procesamiento CSS en el navegador.
- **Filosofía de Cero Dependencias Externas:** El ecosistema visual se maneja de manera puramente declarativa con Tailwind. El proyecto **no cuenta con librerías de componentes ni de animaciones externas** (como Radix UI, Framer Motion o componentes pre-construidos de Shadcn/ui), garantizando un código ligero y libre de sobrecarga (*bloatware*). Las dependencias del archivo [package.json](#) se limitan estrictamente a los núcleos nativos de React, Next.js y el cliente oficial de base de datos.

2.2. Alojamiento y Despliegue (Infraestructura Cloud)

El despliegue del frontend se encuentra centralizado en la plataforma de nube **Vercel**, la cual se integra de forma automatizada con el repositorio de GitHub mediante un flujo de Integración y Despliegue Continuo (CI/CD).

- Toda petición pública es procesada a través de las Redes de Distribución de Contenido (CDN) globales de Vercel.
- El aprovisionamiento de seguridad se gestiona en el borde (*Edge*), garantizando cifrado en tránsito mediante la renovación automática de certificados SSL/TLS bajo un subdominio gratuito corporativo.

2.3. Capa Backend y Persistencia (Supabase BaaS)

La persistencia de datos, la gestión de objetos multimedia y la seguridad lógica se delegan a la plataforma **Supabase**, actuando como un Backend-as-a-Service (BaaS) sobre una base de datos relacional **PostgreSQL**.

- **Cliente de Conexión:** La interacción con el motor de datos se realiza en el frontend mediante un módulo unificado en [src/lib/supabase.ts](#). Este inicializa el cliente

estándar de la librería [@supabase/supabase-js](#) (versión `^2.188.2`), exponiendo la URL del proyecto y la clave pública anónima (*Anon Key*) de forma segura.

- **Estrategia de Sesión:** En el estado actual del desarrollo, la persistencia de sesión se encuentra delegada al cliente tradicional del navegador (`createBrowserClient`). Al no haberse implementado la librería complementaria [@supabase/ssr](#) ni la lógica de un archivo `middleware.ts` en el servidor, el control de acceso y el flujo de tokens de autenticación se gestionan directamente del lado del cliente, agilizando el flujo transaccional directo con la base de datos.

2.4. Estrategia de Carga y Optimización Multimedia

- **Comportamiento del Catálogo Público:** El catálogo en la raíz de la aplicación web se comporta actualmente como una interfaz estática simple. No realiza peticiones masivas automatizadas (*fetches*) repetitivas hacia la base de datos de Supabase en cada carga, lo que protege el consumo de cuotas del plan gratuito (*Free Tier*) de cómputo y previene la degradación de latencia ante el usuario final.
- **Gestión de Imágenes:** El almacenamiento de archivos binarios (fotos de la indumentaria) se concentra en el bucket público de **Supabase Storage**. Las imágenes se consumen de manera directa a través de sus URLs absolutas sin capas intermedias de compresión ni helpers de manipulación en caliente, confiando en la infraestructura nativa de almacenamiento de Supabase para servir los recursos multimedia directo al componente de imagen de Next.js.

Requerimientos F y NF

3. ESPECIFICACIÓN DE REQUERIMIENTOS

El sistema se concibe bajo una estrategia de desarrollo incremental orientada a un Producto Mínimo Viable (MVP). Dado que el repositorio actual cuenta únicamente con una estructura estática base y carece de lógica de negocio o componentes operativos en sus directorios, esta sección define de manera estricta los requerimientos funcionales y no funcionales que guiarán la programación completa de la plataforma desde cero.

3.1. Módulo de Cliente (E-commerce Público)

Este módulo comprende la interfaz expuesta al usuario final para la exploración del catálogo y la preparación del pedido.

3.1.1. Catálogo Dinámico y Filtros Avanzados [● PENDIENTE]

- **RF-01 (Navegación Dinámica):** El sistema debe renderizar las tarjetas de productos consumiendo los datos de la persistencia de forma dinámica, agrupándolos según su categoría correspondiente (ej: Calzas, Remeras, Accesorios).
- **RF-02 (Filtros en el Cliente):** La interfaz debe incluir una barra de selección que permita al usuario filtrar el catálogo por talla, color y categoría de manera combinada, actualizando la vista instantáneamente sin recargar la página.
- **RF-03 (Control de Interfaz por Stock):** Si una variante específica de un artículo (combinación exacta de talla y color) posee una cantidad igual a cero (\$0\$) en la base de datos, el sistema debe deshabilitar automáticamente el botón de compra y mostrar visualmente la etiqueta "Sin Stock".

3.1.2. Gestión de Carrito Local [● PENDIENTE]

- **RF-04 (Persistencia en LocalStorage):** Al no contar con un sistema de usuarios ni bases de datos de clientes, el carrito de compras debe ser gestionado enteramente en el navegador. Se requiere la creación de un estado global o un hook personalizado (`useCart`) que guarde, sume, reste y remueva ítems, persistiendo los datos de manera local a través de la API `localStorage`.

3.1.3. Integración de Checkout vía WhatsApp [● PENDIENTE]

- **RF-05 (Compilación y Redirección Dinámica):** Al accionar el botón de confirmación, el sistema debe desplegar un campo de validación obligatorio para el nombre del cliente. Posteriormente, procesará los elementos activos del `localStorage` y compilará un string sanitizado mediante `encodeURIComponent`, redirigiendo al usuario a una nueva pestaña externa de WhatsApp (`[https://wa.me/](https://wa.me/)`) con un mensaje estructurado listo para enviar.

3.2. Módulo de Administrador (Backoffice Privado)

Este módulo comprende el panel de gestión interna exclusivo para las dos administradoras del comercio (dueña y empleada). Todas las rutas asociadas deberán estructurarse bajo el prefijo `/admin/`.

3.2.1. Autenticación y Control de Accesos [● PENDIENTE]

- **RF-06 (Restricción de Rutas):** El acceso a cualquier vista de administración debe estar estrictamente bloqueado por un formulario de inicio de sesión (Email y Contraseña) conectado al servicio de autenticación. No se debe permitir la visualización de datos confidenciales sin un token válido de sesión.

3.2.2. Panel CRUD de Inventario y Alertas [● PENDIENTE]

- **RF-07 (Operaciones de Stock):** La interfaz debe proveer formularios (**ProductForm**) y tablas (**ProductTable**) funcionales para dar de alta nuevos productos, editar precios, modificar descripciones y actualizar las cantidades disponibles de las variantes físicas de forma directa.
- **RF-08 (Alertas de Stock Crítico):** La tabla de inventario del administrador debe incluir una regla lógica visual que identifique, filtre o resalte (ej. en color rojo) de manera prioritaria aquellos artículos o variantes cuya existencia física sea menor a tres (\$3\$) unidades, permitiendo una rápida toma de decisiones en la reposición de mercadería.

3.2.3. Carga de Medios Integrada [● PENDIENTE]

- **RF-09 (Subida de Archivos):** El formulario de creación de productos debe incluir un componente de entrada de archivos (*File Input*) que capture imágenes desde el dispositivo de la administradora, controle que no superen los 5 MB, las suba directamente al almacenamiento en la nube y asocie la URL resultante al registro del producto correspondiente.

3.2.4. Motor de Conciliación Manual [● PENDIENTE]

- **RF-10 (Deducción Transaccional):** Para mitigar errores de stock, el sistema debe proveer un botón o flujo de "Registrar Venta" dentro del panel. Al ser accionado tras confirmar el pago offline, ejecutará la resta atómica de las unidades exactas vendidas directamente en la matriz de inventario de la base de datos.

3.3. Requerimientos No Funcionales (RNF)

- **RNF-01 (Diseño Orientado a Dispositivos Móviles):** Dado que el catálogo se compartirá en Instagram y el panel de administración se usará activamente en el local, el 100% de las interfaces debe ser completamente responsivo (*Mobile-First*), asegurando botones y elementos interactivos con un tamaño mínimo accesible para pantallas táctiles.
- **RNF-02 (Seguridad por Defecto - RLS):** La base de datos debe implementar políticas estrictas donde los accesos no autenticados queden limitados exclusivamente a la lectura del catálogo activo, blindando cualquier intento de escritura o modificación externa.
- **RNF-03 (Optimización de Recursos y Cuotas):** La arquitectura de componentes debe diseñarse para minimizar las llamadas directas de lectura al motor de base de datos durante las visitas del cliente, protegiendo los límites de transferencia del plan gratuito.

Modelado de datos y BDD

4. MODELO DE DATOS Y ESTADO DE LA BASE DE DATOS

El diseño relacional del sistema migró de un borrador plano hacia una arquitectura normalizada en Tercera Forma Normal (3FN), diseñada específicamente para garantizar la integridad referencial y optimizar las consultas concurrentes dentro de los límites del plan gratuito de Supabase.

4.1. Evolución del Esquema Relacional [● COMPLETADO]

A diferencia del diseño teórico inicial planteado en los primeros bocetos (donde las categorías se manejaban como texto libre e inglés), el esquema definitivo desplegado e inyectado en la instancia real de PostgreSQL de Supabase adopta una estructura normalizada y unificada en idioma español. Esta decisión mitiga inconsistencias de escritura entre las administradoras y asegura la escalabilidad del catálogo.

4.2. Diccionario de Datos (Entidades Vigentes)

4.2.1. Tabla: **categorias** [● COMPLETADO]

Almacena las agrupaciones principales de las prendas para estructurar la navegación del e-commerce y los filtros del panel. Está poblada con las 7 categorías reales del negocio: *Remeras, Tops, Buzos y Camperas, Calzas, Shorts y Bermudas, Joggings, Accesorios*.

- **id** (UUID, Primary Key): Identificador único generado automáticamente mediante `gen_random_uuid()`.
- **nombre** (VARCHAR, Not Null): Denominación visible de la categoría (Ej: "Buzos y Camperas").
- **slug** (VARCHAR, Unique, Not Null): Identificador amigable para la construcción de filtros en las URLs del navegador (Ej: `buzos-y-camperas`).
- **created_at** (TIMESTAMPTZ): Registro de auditoría temporal con zona horaria por defecto `now()`.

4.2.2. Tabla: **productos** [● EN DESARROLLO]

Define la entidad genérica de la indumentaria, abstrayendo las propiedades comerciales comunes.

- **id** (UUID, Primary Key): Identificador único indexado.
- **nombre** (VARCHAR, Not Null): Nombre comercial del artículo.
- **descripcion** (TEXT, Nullable): Detalle de la composición de la tela o instrucciones de lavado.
- **categoria_id** (UUID, Foreign Key): Vinculación relacional directa hacia `categorias.id` bajo la restricción `ON DELETE RESTRICT` (impide dejar productos huérfanos si se borra una categoría).
- **imagenes** (TEXT[]): Array de URLs absolutas que apuntan a los archivos binarios guardados en Supabase Storage.

- **activo** (BOOLEAN, default true): Flag lógico diseñado para soportar el patrón de borrado lógico (*Soft Delete*), ocultando el producto del catálogo público sin romper el historial de ventas del administrador.
- **created_at** (TIMESTAMPTZ): Marca de tiempo automática.
- *Nota de Deuda Técnica*: La columna heredada **categoría** (texto libre) aún existe con restricción **NOT NULL** en la base de datos. Representa un riesgo de bloqueo para los nuevos formularios del frontend y está marcada para ser deprecada o eliminada.

4.2.3. Tabla: **variantes_stock** [● COMPLETADO]

Entidad débil encargada de resolver la matriz multidimensional de inventario (Talle × Color × Cantidad) de forma atómica. El precio se modela explícitamente en esta tabla para permitir costos diferenciados por talle o color en una misma prenda (ej. mayor consumo de tela en talles grandes).

- **id** (UUID, Primary Key): Identificador atómico.
- **producto_id** (UUID, Foreign Key): Vinculación con **productos.id** configurada con **ON DELETE CASCADE** (si un modelo se discontinúa y elimina, todas sus variantes físicas se limpian en cascada).
- **talle** (VARCHAR, Not Null): Escala de talle de la prenda (Ej: "S", "M", "L", "42").
- **color** (VARCHAR, Not Null): Atributo cromático del artículo (Ej: "Negro", "Fucsia").
- **cantidad** (INTEGER, default 0): Inventario físico disponible en estantería. Por regla de negocio, este valor debe ser siempre de grado mayor o igual a cero (≥ 0).
- **precio** (NUMERIC): Valor monetario parametrizado para soportar fluctuaciones económicas.

4.3. Estrategia de Indexación y Optimización [● COMPLETADO]

Para garantizar que el sistema mantenga una latencia baja dentro de las cuotas gratuitas de cómputo de Supabase, se inyectaron índices específicos en el motor PostgreSQL:

- **Índice B-Tree sobre categorías(slug)**: Optimiza las búsquedas instantáneas cuando el cliente filtra productos interactuando con la URL del e-commerce.
- **Índice B-Tree sobre productos(categoría_id)**: Acelera los mapeos y operaciones de cruce (*joins*) entre las tablas de productos y sus categorías.
- **Índice Compuesto sobre variantes_stock(producto_id, talle, color)**: Diseñado específicamente para que el frontend consulte a velocidad extrema si una combinación particular de talle y color tiene stock disponible antes de habilitar el botón de compra en la interfaz del cliente.

Seguridad e Infraestructura

5. SEGURIDAD Y CONFIGURACIÓN DE INFRAESTRUCTURA

Esta sección detalla los mecanismos de aislamiento de datos, las reglas de acceso perimetral y la arquitectura del espacio de trabajo local. Siguiendo el principio de menor privilegio, la seguridad se delega directamente en el motor de la base de datos y la infraestructura cloud, garantizando un entorno robusto e inmune a manipulaciones desde el frontend.

5.1. Políticas de Seguridad a Nivel de Fila (RLS) [●] COMPLETADO]

El aislamiento y la protección de los datos transaccionales no dependen de la lógica del cliente, sino de reglas imperativas inyectadas en PostgreSQL mediante el script `schema.sql`. La seguridad a nivel de fila (Row Level Security - RLS) se encuentra activa y operativa en la instancia de Supabase para las tres tablas del sistema:

SQL

```
ALTER TABLE categorias ENABLE ROW LEVEL SECURITY;  
ALTER TABLE productos ENABLE ROW LEVEL SECURITY;  
ALTER TABLE variantes_stock ENABLE ROW LEVEL SECURITY;
```

Las políticas de acceso diferencian estrictamente los dos roles de la plataforma:

- **Rol Público / Anónimo (anon):** Posee permisos exclusivos de lectura básica (`SELECT`). Cualquier usuario que ingrese a la web puede consumir el catálogo de productos y categorías sin necesidad de autenticarse.
- **Rol Administrativo / Autenticado (authenticated):** Exclusivo para las operadoras del negocio (dueña y empleada). Requiere un JSON Web Token (JWT) válido provisto por Supabase Auth para habilitar operaciones de escritura, edición y borrado (`INSERT`, `UPDATE`, `DELETE`).

5.2. Infraestructura de Storage (Bucket de Medios) [●] COMPLETADO]

Para el alojamiento persistente de las imágenes de la indumentaria, se configuró y desplegó un contenedor de objetos binarios en Supabase Storage con las siguientes especificaciones técnicas:

- **Identificador del Bucket:** `productos-imagenes` (referenciado de forma exacta en los componentes de carga del frontend).
- **Nivel de Visibilidad:** **PÚBLICO**. Esto permite que el método `getPublicUrl()` exponga los recursos multimedia directamente al e-commerce mediante enlaces absolutos, acelerando el renderizado en el navegador.
- **Restricciones de Carga:** Límite máximo de tamaño por archivo parametrizado en **5 MB**.

- **Políticas de Seguridad en Storage:** Al igual que las tablas, el almacenamiento público restringe la mutación de archivos. La lectura es libre para el catálogo, pero las operaciones de inserción y borrado de imágenes requieren explícitamente que la solicitud provenga de un usuario autenticado bajo el rol de administrador.

5.3. Control de Acceso en el Servidor (Middleware) [●] PENDIENTE]

- **Estado Actual:** En el repositorio actual **no existe un archivo `middleware.ts`** encargado de interceptar las peticiones a nivel de servidor en Next.js.
- **Estrategia de Implementación:** Al no contar con lógica de control de cookies o sesión centralizada en el servidor, las rutas privadas `/admin/*` se encuentran temporalmente expuestas en el frontend. La protección de rutas deberá programarse desde cero en las próximas fases, utilizando la suite de herramientas intermitentes del lado del cliente o incorporando un proxy de autenticación intermedio.

5.4. Configuración del Espacio de Trabajo Local [●] COMPLETADO]

El proyecto se encuentra inicializado localmente bajo una estructura limpia de monorepo simplificado, implementando dos mecanismos críticos para la protección de secretos de desarrollo:

- **Aislamiento de Credenciales (`web/.env.example`):** Archivo calco que actúa como plantilla pública en el repositorio de GitHub. Permite clonar la estructura de variables de entorno requeridas (`NEXT_PUBLIC_SUPABASE_URL` y `NEXT_PUBLIC_SUPABASE_ANON_KEY`) sin exponer las claves reales del proyecto en producción.
- **Exclusiones de Git (`web/.gitignore`):** Archivo de directivas de exclusión encargado de blindar el entorno local. Evita que archivos temporales de compilación (`.next/`, `out/`, `node_modules/`) o archivos con las credenciales reales del negocio (`.env`, `.env.local`) se filtren accidentalmente al entorno público de internet.

Modulos criticos y logica de negocio

6. ESPECIFICACIÓN DE MÓDULOS CRÍTICOS Y LÓGICA DE NEGOCIO

Esta sección describe la ingeniería detrás de los dos componentes más complejos del sistema. Aunque el frontend se encuentra actualmente en una etapa estática inicial sin estas funciones codificadas, las especificaciones analizadas a continuación establecen el estándar algorítmico y de flujo que deberá ejecutarse en la fase de desarrollo.

6.1. Algoritmo de Checkout para WhatsApp [● PENDIENTE]

El núcleo transaccional del e-commerce público se basa en una redirección dinámica asincrónica. El objetivo de este módulo es procesar el estado intermedio del carrito guardado en el navegador, sanitizar la entrada de datos del usuario y transformarla en un canal de comunicación directo con la administradora.

6.1.1. Flujo Lógico del Componente

Cuando el cliente decide finalizar la orden, la aplicación debe ejecutar de forma secuencial los siguientes pasos lógicos:

1. **Validación de Identidad:** Validar que el campo del nombre del cliente no esté vacío y aplicar un método `.trim()` para eliminar espacios redundantes.
2. **Procesamiento del Carrito:** Iterar sobre el array de objetos recuperado del `localStorage` (donde cada ítem contiene las propiedades de nombre, talla, color y cantidad).
3. **Construcción del Mensaje:** Compilar una cadena de texto (*string*) con formato estructurado que actúe como una "orden de compra" legible para el comercio.
4. **Codificación URI:** Aplicar la función nativa `encodeURIComponent()` sobre el mensaje completo para asegurar la compatibilidad internacional de caracteres, espacios y saltos de línea dentro de la URL.
5. **Redirección Segura:** Invocar el método `window.open()` apuntando al protocolo de la API de WhatsApp Business (`[https://wa.me/](https://wa.me/)` junto al número comercial parametrizado), utilizando el flag `_blank` para no destruir el estado ni la pestaña activa de la web del usuario.

6.1.2. Plantilla del Script de Redirección (Propuesta Técnica)

El agente autónomo o el desarrollador deberán implementar una estructura funcional equivalente a la siguiente lógica en JavaScript/TypeScript:

JavaScript

```
const procesarCheckoutWEEKSPORT = (nombreCliente, itemsCarrito) => {  
  const TELEFONO_COMERCIAL = "54911XXXXXXXXX"; // Número asignado a  
  WEEKSPORT
```

```
  // Encabezado institucional del mensaje
```

```
let cuerpoMensaje = `¡Hola!, mi nombre es ${nombreCliente.trim()} y estoy interesado/a en  
estos productos de WEEKSPORT:\n\n`;
```

```
// Construcción iterativa del listado de productos  
itemsCarrito.forEach((item) => {  
  cuerpoMensaje += ` • ${item.nombre} (Talle: ${item.talle}, Color: ${item.color})  
x${item.cantidad}\n`;  
});
```

```
cuerpoMensaje += `\n¿Podrías darme detalles así pactamos el pago y la entrega del  
pedido?`; //
```

```
// Aplicación de URL Encoding estándar  
const textoCodificado = encodeURIComponent(cuerpoMensaje);
```

```
// Ejecución de la redirección en pestaña nueva[cite: 2]  
window.open(`https://wa.me/${TELEFONO_COMERCIAL}?text=${textoCodificado}`,  
'_blank');  
};
```

6.2. Arquitectura de Autenticación y Gestión de Sesiones [● PENDIENTE]

El acceso a las vistas de administración de inventario requiere un esquema de validación blindado para evitar la manipulación de stock por parte de usuarios externos.

6.2.1. Mitigación del Conflicto de Persistencia (Estrategia)

Durante las auditorías de diseño previas, se identificó un riesgo técnico común en arquitecturas híbridas de Next.js: el conflicto de lectura de sesión entre cliente y servidor[cite: 2]. Si el frontend inicia sesión utilizando el cliente clásico de Supabase en el navegador, las credenciales se guardan de forma interna en el **localStorage**, volviéndose invisibles para los archivos que se ejecutan del lado del servidor (como las rutas protegidas o los componentes SSR)[cite: 2].

Para asegurar una arquitectura limpia y profesional, la lógica de autenticación deberá seguir estrictamente una estrategia unificada[cite: 2]:

- **Sincronización mediante Cookies:** Se descarta el almacenamiento exclusivo en **localStorage** para datos críticos de sesión.
- **Unificación de Clientes:** Toda validación de sesión en las rutas **/admin/*** deberá configurarse utilizando herramientas que sincronicen de manera automática los tokens basados en JSON Web Tokens (JWT) mediante cookies seguras, asegurando que el estado del usuario sea exactamente el mismo tanto en los componentes de cliente como en los mecanismos de control del servidor[cite: 2].
- **Suscripción Activa:** El panel administrativo deberá implementar un escuchador global (**onAuthStateChange**) para detectar la expiración o el cierre de sesión de

forma inmediata, bloqueando la pantalla y redirigiendo al formulario de login sin demoras[cite: 2].

QA y auditorías

7. CONTROL DE CALIDAD (QA) Y REGISTRO DE AUDITORÍA

Esta sección establece los mecanismos de control aplicados para garantizar la estabilidad técnica del entorno inicial del proyecto, junto con el registro histórico de las sesiones de desarrollo, auditorías de código y errores críticos subsanados.

7.1. Matriz de Verificación (Casos de Prueba) [●] COMPLETADO]

Para certificar la integridad estructural antes de iniciar la programación masiva de componentes visuales de la Fase 3, se ejecutó una matriz de control de calidad sobre el repositorio local y la instancia de Supabase, obteniendo una validación absoluta de los cimientos del sistema:

Identificador	Prueba de Verificación	Método de Control	Resultado Esperado	Estado Actual
TC-01	Ejecución del Servidor Local	Correr comando <code>cd web && npm run dev</code> en terminal.	Compilación limpia de Next.js. Acceso a http://localhost:3000 sin errores en la consola.	VALIDADO
TC-02	Auditoría de Aislamiento de Credenciales	Inspección mediante comando <code>git status</code> y revisión de directivas.	Confirmar que los archivos <code>.env.local</code> con claves reales permanecen ignorados por Git y no se filtran.	VALIDADO
TC-03	Integridad Estructural SQL	Inyección del script <code>schema.sql</code> en el SQL Editor de Supabase.	Creación exitosa de las 3 tablas relacionales (<code>categorias</code> , <code>productos</code> , <code>variantes_stock</code>) con RLS e índices	VALIDADO

			activos, sin fallos de sintaxis.	
--	--	--	----------------------------------	--

7.2. Registro Histórico de Sesiones de Desarrollo [● COMPLETADO]

A fin de mantener la trazabilidad de la evolución del software, se documenta la metodología de trabajo híbrida aplicada durante las fases tempranas del ciclo de vida del proyecto:

- **Intervención de Agentes Autónomos:** El andamiaje inicial del frontend y la configuración de tipos fue delegada a un agente de codificación automatizado (Antigravity) que operó sobre el repositorio local generando 17 archivos base. Dicho proceso fue interrumpido temporalmente por agotamiento de cuotas de API, dejando piezas del panel administrativo incompletas y sin verificar.
- **Auditoría y Sincronización:** Se ejecutó un proceso de diagnóstico manual en este espacio de trabajo para contrastar el diseño teórico con el código funcional. Esto permitió detectar desvíos críticos entre los esquemas (como el uso original de nomenclatura en inglés en el DERA teórico frente a las tablas reales desplegadas en español en la base de datos de producción), unificando el criterio técnico bajo la nomenclatura real en español para garantizar la consistencia.

7.3. Registro de Errores Resueltos (Log de Bugs) [● COMPLETADO]

Durante la consolidación de la estructura del proyecto, se identificaron y resolvieron los siguientes bloqueos técnicos y divergencias de arquitectura:

7.3.1. Colisión de Rutas en la Raíz (/)

- **Síntoma:** Al levantar el servidor, el navegador renderizaba un mensaje estático de "Plataforma en construcción", volviendo invisible el catálogo funcional de productos.
- **Causa Raíz:** Coexistían en el App Router los archivos `src/app/page.tsx` (marcador de posición) y `src/app/(public)/page.tsx` (catálogo real conectados a Supabase). Next.js resolvía la ruta raíz priorizando el archivo de la jerarquía superior.
- **Solución:** Eliminación definitiva del archivo placeholder estático en la raíz para permitir el enrutamiento limpio hacia el catálogo real.

7.3.2. Error de Compilación en `Footer.tsx` (Server vs. Client Components)

- **Síntoma:** El compilador de Next.js arrojaba la excepción *"Event handlers cannot be passed to Client Component props"*.
- **Causa Raíz:** El componente `Footer.tsx` incluía manejadores de eventos interactivos de React (`onMouseEnter` y `onMouseLeave`), pero estaba siendo renderizado dentro de un componente de servidor (`(public)/layout.tsx`) sin declarar su naturaleza interactiva.
- **Solución:** Inyección de la directiva `'use client'` en la primera línea de `Footer.tsx` para forzar su correcta hidratación en el cliente.

7.3.3. Migración de Categorías Hardcodeadas a Relacionales

- **Síntoma:** Riesgo de inconsistencia de datos al mantener arrays de texto estáticos en el frontend para filtrar calzas, remeras o accesorios.
- **Causa Raíz:** El código dependía del tipo TypeScript estático `Categoria` y de una constante fija denominada `CATEGORIAS`, aislando el frontend de la base de datos.
- **Solución:** Refactorización integral de 9 archivos críticos (incluyendo `FilterBar.tsx`, `ProductForm.tsx` y `ProductCard.tsx`) para reemplazar las constantes fijas por la interfaz dinámica `CategoriaDB`, conectando las vistas de manera directa con la tabla de PostgreSQL mediante operaciones de unión (*joins*).

7.3.4. Bucle Infinito de Redirección en el Acceso `/admin/login`

- **Síntoma:** Tras introducir credenciales válidas en el panel administrativo, la red confirmaba un estado de login exitoso (HTTP 200), pero el sistema entraba en un bucle infinito de redirecciones entre `/admin` y `/admin/login`.
- **Causa Raíz:** El cliente de base de datos se inicializaba con `createClient` tradicional (guardando la sesión en `localStorage`), mientras que el middleware del servidor evaluaba los accesos con `createServerClient` (buscando cookies). Al ser mecanismos incompatibles, el servidor no podía leer la sesión iniciada por el navegador.
- **Solución:** Reemplazo de la instanciación en el cliente por `createBrowserClient`, unificando el ecosistema de persistencia de tokens para que el servidor valide las cookies de forma fluida.

Tareas pendientes

8. ROADMAP Y PLAN DE ACCIÓN HACIA EL MVP (TAREAS PENDIENTES)

Esta sección establece la hoja de ruta estratégica y el desglose de tareas técnicas necesarias para concluir la Fase 1 y consolidar el Producto Mínimo Viable (MVP) de la plataforma. Al mapear con precisión la deuda técnica y los pasos pendientes, el proyecto mantiene un flujo de desarrollo predecible y ordenado.

8.1. Resolución del Bug Bloqueante Inmediato [🟡 EN DESARROLLO]

El principal obstáculo técnico al cierre de la última sesión de consolidación radica en el flujo final del asistente de creación de productos (`ProductForm.tsx`). Tras verificar que la carga de archivos multimedia en Supabase Storage funciona de manera correcta, el paso final de inserción en la base de datos es rechazado por el motor PostgreSQL.

- **Diagnóstico del Problema:** La tabla `productos` conserva una columna heredada denominada `categoria` configurada con la restricción `NOT NULL`. Debido a que el nuevo formulario del frontend fue refactorizado con éxito para enviar exclusivamente el identificador relacional `categoria_id` (UUID), la sentencia `INSERT` no incluye ningún valor para el viejo campo de texto libre, provocando que la base de datos aborte la transacción.
- **Plan de Acción Técnico:** Se procederá a confirmar el mensaje exacto de excepción inspeccionando la respuesta en la pestaña *Network* del navegador. La resolución se ejecutará aplicando un script de alteración de tabla en el panel de Supabase mediante cualquiera de estas dos vías sugeridas:

SQL

-- Opción A: Flexibilizar la columna inerte para no bloquear el frontend
`ALTER TABLE productos ALTER COLUMN categoria DROP NOT NULL;`

-- Opción B: Remoción definitiva tras confirmar que no existen referencias activas
`ALTER TABLE productos DROP COLUMN categoria;`

8.2. Verificación y Pruebas End-to-End (E2E) [🔴 PENDIENTE]

Una vez subsanado el conflicto del formulario de alta, se requiere ejecutar un ciclo completo de control de calidad manual para validar la integración de los componentes del sistema:

- **Verificación del Catálogo:** Certificar que un producto creado desde el panel administrativo renderice instantáneamente en la interfaz pública del cliente, respetando sus categorías e imágenes.
- **Persistencia de Variantes:** Validar que el formulario de edición cargue correctamente las matrices de talla, color, stock y precio previamente registradas para cada prenda.

- **Prueba del Ciclo de Descuento (RF-10):** Probar el componente de validación de ventas (`SaleValidator`) en el backoffice para asegurar que, al confirmar un pago manual, la transacción descuente con precisión matemática las unidades físicas en la tabla `variantes_stock`.

8.3. Refactorización y Limpieza de Código [● PENDIENTE]

A fin de adecuar la estructura del software a las mejores prácticas de mantenibilidad y estándares de la industria, se contemplan las siguientes tareas de refactorización:

- **Migración de Middleware:** Adaptar el archivo de interceptación de rutas `middleware.ts` hacia la convención de configuración basada en proxies, anticipando futuras actualizaciones y optimizando la lectura de cookies en el servidor.
- **Depuración del Modelo Relacional:** Confirmar la ausencia total de referencias antiguas hacia el campo `productos.categoria` en los componentes del frontend para ejecutar su remoción definitiva del motor de base de datos.

8.4. Preparación para el Despliegue de Producción (Go-Live) [● PENDIENTE]

El paso final para la salida a producción de la plataforma familiar consiste en sustituir los componentes genéricos y las credenciales de prueba por los activos oficiales del negocio:

- **Identidad de Marca:** Reemplazar el logotipo provisorio del encabezado e implementar la paleta de colores oficial de la marca (sustituyendo los tonos fucsia y naranja utilizados como referencia de desarrollo).
- **Canal Comercial Real:** Sustituir el número de teléfono personal configurado en el script de checkout por la línea comercial de WhatsApp Business de la Community Manager de la tienda.
- **Población del Inventario:** Coordinar con las administradoras la transición del control de stock desde la hoja de cálculo de Google Sheets hacia el panel administrativo de la aplicación, cargando el catálogo inicial con descripciones, precios reales por variante y fotografías optimizadas.